

Aim: Supervised Learning Algorithm for Predictive Maintenance

Resources needed: Artificial Intelligence Documentation, AIMA libraries, Data Collection,

Pre Lab/ Prior Concepts:

Students should have prior knowledge of Basic Programming skills and Fundamentals of AI and Machine Learning.

Theory:

Machine learning is a branch of Artificial Intelligence that focuses on developing models and algorithms that let computers learn from data without being explicitly programmed for every task. In simple words, ML teaches systems to think and understand like humans by learning from the data.

Machine Learning is mainly divided into three core types: Supervised, Unsupervised and Reinforcement Learning, along with two additional types, Semi-Supervised and Self-Supervised Learning.

Supervised Learning: Trains models on labelled data to predict or classify new, unseen data. Unsupervised Learning: Finds patterns or groups in unlabelled data, like clustering or dimensionality reduction.

Reinforcement Learning: Learns through trial and error to maximise rewards, ideal for decision-making tasks.

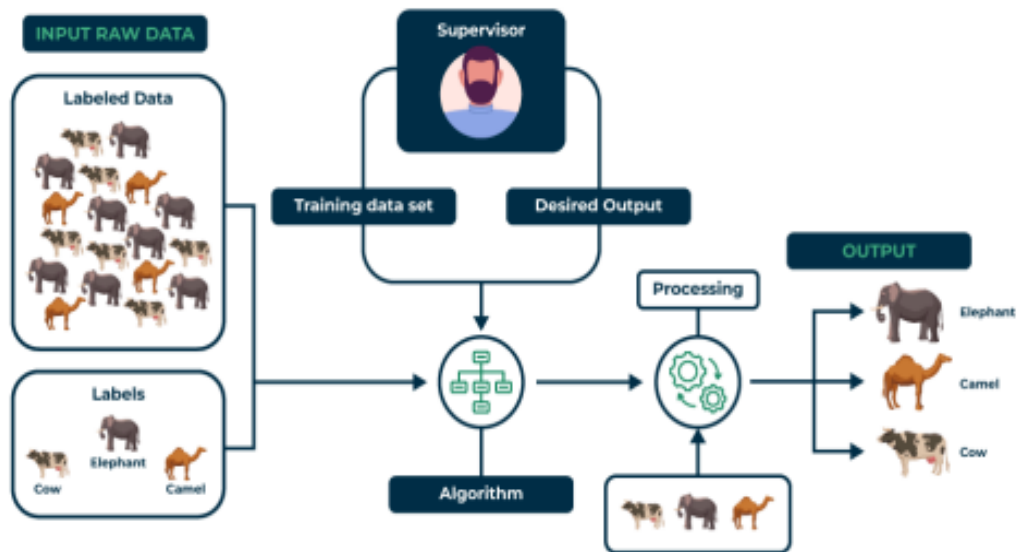
Supervised Learning

Supervised learning algorithms are generally categorised into **two main types:**

Classification - where the goal is to predict discrete labels or categories

Regression - where the aim is to predict continuous numerical values.

Supervised Learning



There are many algorithms used in supervised learning, each suited to different types of problems. Some of the most commonly used supervised learning algorithms are:

1. Linear Regression

This is one of the simplest ways to predict numbers using a straight line. It helps find the relationship between input and output.

- Linear Regression
- Gradient Descent in Linear Regression
- Multiple Linear Regression
- Ridge Regression
- Lasso regression
- Elastic net Regression

Using Linear regression, find the accuracy.

2. Develop regression and classification models (Linear regression) to predict machine failure or maintenance schedules based on historical data.

Code:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet, LogisticRegression
from sklearn.preprocessing import StandardScaler
```

```

from sklearn.metrics import mean_squared_error, r2_score,
accuracy_score, confusion_matrix, classification_report,
mean_absolute_error
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
plt.style.use('seaborn-v0_8-darkgrid')
print("="*80)
print("EXPERIMENT NO. 2: SUPERVISED LEARNING FOR PREDICTIVE
MAINTENANCE")
print("="*80)
data =
pd.read_csv('/media/galdrux/galdrux_storage/sem_5/AIR/dataset/battery_f
eatures.csv')
data.columns = ['cycle', 'mean_voltage', 'min_voltage', 'max_voltage',
'mean_current', 'mean_temperature', 'max_temperature',
'discharge_time', 'capacity']
print("\nDATASET OVERVIEW")
print("="*80)
print(f"Dataset Shape: {data.shape[0]} rows, {data.shape[1]} columns")
print("\nFirst 5 rows:")
print(data.head())
print("\nStatistical Summary:")
print(data.describe())
data['voltage_range'] = data['max_voltage'] - data['min_voltage']
data['temp_range'] = data['max_temperature'] - data['mean_temperature']
data['power'] = data['mean_voltage'] * abs(data['mean_current'])
initial_capacity = data['capacity'].iloc[0]
failure_threshold = 0.7 * initial_capacity
data['failure'] = (data['capacity'] < failure_threshold).astype(int)
print("\nFEATURE ENGINEERING")
print("="*80)
print(f"Initial Capacity: {initial_capacity:.4f} Ah")
print(f"Failure Threshold: {failure_threshold:.4f} Ah (70% of
initial)")
print(f"Total Failure Cycles: {data['failure'].sum()}")
print(f"Failure Rate: {data['failure'].mean()*100:.2f}%")
print("\n" + "="*80)
print("PART 1: LINEAR REGRESSION MODELS FOR CAPACITY PREDICTION")
print("="*80)

```

```

feature_cols = ['cycle', 'mean_voltage', 'min_voltage', 'max_voltage',
'mean_current',          'mean_temperature',          'max_temperature',
'discharge_time', 'voltage_range', 'temp_range', 'power']
X = data[feature_cols]
y_reg = data['capacity']
X_train, X_test, y_train, y_test = train_test_split(X, y_reg,
test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print("\n1.1 SIMPLE LINEAR REGRESSION (Cycle vs Capacity)")
X_simple = data[['cycle']]
y_simple = data['capacity']
X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(X_simple,
y_simple, test_size=0.2, random_state=42)
simple_lr = LinearRegression()
simple_lr.fit(X_train_s, y_train_s)
y_pred_s = simple_lr.predict(X_test_s)
mse_s = mean_squared_error(y_test_s, y_pred_s)
r2_s = r2_score(y_test_s, y_pred_s)
mae_s = mean_absolute_error(y_test_s, y_pred_s)
print(f"Equation: Capacity = {simple_lr.coef_[0]:.6f} * Cycle +
{simple_lr.intercept_:.6f}")
print(f"R² Score (Accuracy): {r2_s:.6f}")
print(f"MSE: {mse_s:.6f}")
print(f"MAE: {mae_s:.6f}")
print("\n1.2 MULTIPLE LINEAR REGRESSION")
mlr = LinearRegression()
mlr.fit(X_train_scaled, y_train)
y_pred_ml = mlr.predict(X_test_scaled)
mse_ml = mean_squared_error(y_test, y_pred_ml)
r2_ml = r2_score(y_test, y_pred_ml)
mae_ml = mean_absolute_error(y_test, y_pred_ml)
print(f"R² Score (Accuracy): {r2_ml:.6f}")
print(f"MSE: {mse_ml:.6f}")
print(f"MAE: {mae_ml:.6f}")
print("\n1.3 RIDGE REGRESSION")
ridge = Ridge(alpha=1.0)
ridge.fit(X_train_scaled, y_train)
y_pred_ridge = ridge.predict(X_test_scaled)
r2_ridge = r2_score(y_test, y_pred_ridge)
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
print(f"R² Score (Accuracy): {r2_ridge:.6f}")

```

```

print(f"MSE: {mse_ridge:.6f}")
print("\n1.4 LASSO REGRESSION")
lasso = Lasso(alpha=0.01)
lasso.fit(X_train_scaled, y_train)
y_pred_lasso = lasso.predict(X_test_scaled)
r2_lasso = r2_score(y_test, y_pred_lasso)
mse_lasso = mean_squared_error(y_test, y_pred_lasso)
print(f"R2 Score (Accuracy): {r2_lasso:.6f}")
print(f"MSE: {mse_lasso:.6f}")
print("\n1.5 ELASTIC NET REGRESSION")
elastic = ElasticNet(alpha=0.1, l1_ratio=0.5)
elastic.fit(X_train_scaled, y_train)
y_pred_elastic = elastic.predict(X_test_scaled)
r2_elastic = r2_score(y_test, y_pred_elastic)
mse_elastic = mean_squared_error(y_test, y_pred_elastic)
print(f"R2 Score (Accuracy): {r2_elastic:.6f}")
print(f"MSE: {mse_elastic:.6f}")
print("\n1.6 GRADIENT DESCENT IMPLEMENTATION")
class GradientDescentLR:
    def __init__(self, learning_rate=0.01, iterations=1000):
        self.lr = learning_rate
        self.iterations = iterations
        self.weights = None
        self.bias = None
        self.cost_history = []
    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0
        self.cost_history = []
        for iteration in range(self.iterations):
            y_pred = np.dot(X, self.weights) + self.bias
            dw = (1/n_samples) * np.dot(X.T, (y_pred - y))
            db = (1/n_samples) * np.sum(y_pred - y)
            self.weights -= self.lr * dw
            self.bias -= self.lr * db
            cost = (1/(2*n_samples)) * np.sum((y_pred - y) ** 2)
            self.cost_history.append(cost)
            if iteration % 200 == 0:
                print(f"Iteration {iteration}: Cost = {cost:.6f}")
        return self
    def predict(self, X):
        return np.dot(X, self.weights) + self.bias

```

```

gd = GradientDescentLR(learning_rate=0.01, iterations=1000)
gd.fit(X_train_scaled, y_train.values)
y_pred_gd = gd.predict(X_test_scaled)
r2_gd = r2_score(y_test, y_pred_gd)
mse_gd = mean_squared_error(y_test, y_pred_gd)
print(f"\nR2 Score (Accuracy): {r2_gd:.6f}")
print(f"MSE: {mse_gd:.6f}")
print("\nREGRESSION MODELS COMPARISON")
print("="*80)
comparison = pd.DataFrame({
    'Model': ['Simple Linear', 'Multiple Linear', 'Ridge', 'Lasso',
'Elastic Net', 'Gradient Descent'],
    'R2_Score': [r2_s, r2_mlr, r2_ridge, r2_lasso, r2_elastic, r2_gd],
    'MSE': [mse_s, mse_mlr, mse_ridge, mse_lasso, mse_elastic, mse_gd]
})
print(comparison.to_string(index=False))
best_model = comparison.loc[comparison['R2_Score'].idxmax()]
print(f"\nBEST REGRESSION MODEL: {best_model['Model']}")
print(f"R2 Score: {best_model['R2_Score']:.6f}")
print(f"MSE: {best_model['MSE']:.6f}")
plt.figure(figsize=(14, 5))
plt.subplot(1, 2, 1)
models = comparison['Model']
r2_scores = comparison['R2_Score']
plt.bar(models, r2_scores, color=['blue', 'green', 'red', 'purple',
'orange', 'cyan'])
plt.xlabel('Regression Models')
plt.ylabel('R2 Score (Accuracy)')
plt.title('R2 Score Comparison of Regression Models')
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)
plt.subplot(1, 2, 2)
plt.scatter(y_test, y_pred_mlr, alpha=0.6, label='Predictions')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
'r--', label='Perfect Prediction', linewidth=2)
plt.xlabel('Actual Capacity (Ah)')
plt.ylabel('Predicted Capacity (Ah)')
plt.title(f'Multiple Linear Regression\nActual vs Predicted (R2 =
{r2_mlr:.4f})')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('regression_accuracy.png', dpi=150, bbox_inches='tight')

```

```

plt.show()
plt.figure(figsize=(10, 4))
plt.plot(gd.cost_history)
plt.xlabel('Iteration')
plt.ylabel('Cost (MSE)')
plt.title('Gradient Descent Cost Function Convergence')
plt.grid(True, alpha=0.3)
plt.savefig('gradient_descent.png', dpi=150, bbox_inches='tight')
plt.show()
print("\n" + "="*80)
print("PART 2: CLASSIFICATION MODEL FOR FAILURE PREDICTION")
print("="*80)
X_clf = data[feature_cols]
y_clf = data['failure']
X_train_clf, X_test_clf, y_train_clf, y_test_clf =
train_test_split(X_clf, y_clf, test_size=0.2, random_state=42)
scaler_clf = StandardScaler()
X_train_clf_scaled = scaler_clf.fit_transform(X_train_clf)
X_test_clf_scaled = scaler_clf.transform(X_test_clf)
log_reg = LogisticRegression(random_state=42, max_iter=1000)
log_reg.fit(X_train_clf_scaled, y_train_clf)
y_pred_clf = log_reg.predict(X_test_clf_scaled)
y_prob_clf = log_reg.predict_proba(X_test_clf_scaled)[:, 1]
accuracy = accuracy_score(y_test_clf, y_pred_clf)
cm = confusion_matrix(y_test_clf, y_pred_clf)
print(f"Classification Accuracy: {accuracy:.6f}")
print(f"\nConfusion Matrix:")
print(cm)
print(f"\nClassification Report:")
print(classification_report(y_test_clf, y_pred_clf, target_names=['No
Failure', 'Failure']))
from sklearn.metrics import roc_curve, roc_auc_score
fpr, tpr, thresholds = roc_curve(y_test_clf, y_prob_clf)
auc_score = roc_auc_score(y_test_clf, y_prob_clf)
print(f"\nROC-AUC Score: {auc_score:.6f}")
feature_importance = pd.DataFrame({
    'Feature': feature_cols,
    'Coefficient': log_reg.coef_[0]
}).sort_values('Coefficient', key=abs, ascending=False)
print("\nTOP 5 IMPORTANT FEATURES FOR FAILURE PREDICTION:")
print(feature_importance.head(5).to_string(index=False))
plt.figure(figsize=(15, 5))
plt.subplot(1, 3, 1)

```

```

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['No
Failure', 'Failure'], yticklabels=['No Failure', 'Failure'])
plt.title('Confusion Matrix - Logistic Regression')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.subplot(1, 3, 2)
plt.plot(fpr, tpr, label=f'Logistic Regression (AUC =
{auc_score:.3f})', linewidth=2)
plt.plot([0, 1], [0, 1], 'r--', label='Random Classifier', linewidth=2)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.grid(True, alpha=0.3)
plt.subplot(1, 3, 3)
plt.barh(feature_importance.head(5) ['Feature'],
feature_importance.head(5) ['Coefficient'].abs(), color='red',
alpha=0.7)
plt.xlabel('|Coefficient|')
plt.title('Top 5 Features for Failure Prediction')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('classification_results.png', dpi=150, bbox_inches='tight')
plt.show()
print("\n" + "="*80)
print("PART 3: PREDICTIVE MAINTENANCE SCHEDULING")
print("="*80)
future_cycles = np.arange(data['cycle'].max() + 1, data['cycle'].max()
+ 21)
print(f"Predicting for next 20 cycles: {future_cycles}")
recent_data = data.tail(30)
future_features = []
for i, cycle in enumerate(future_cycles):
    feature_vector = []
    for col in feature_cols:
        if col == 'cycle':
            feature_vector.append(cycle)
        else:
            recent_mean = recent_data[col].mean()
            if col in ['mean_voltage', 'min_voltage', 'max_voltage']:
                feature_vector.append(recent_mean - 0.001 * (i+1))
            elif col in ['mean_temperature', 'max_temperature']:
                feature_vector.append(recent_mean + 0.05 * (i+1))

```

```

        elif col == 'discharge_time':
            feature_vector.append(recent_mean - 0.3 * (i+1))
        else:
            feature_vector.append(recent_mean)
    future_features.append(feature_vector)
future_X = np.array(future_features)
future_X_scaled = scaler.transform(future_X)
future_capacity = mlr.predict(future_X_scaled)
future_failure_prob =
log_reg.predict_proba(scaler_clf.transform(future_X))[:, 1]
future_df = pd.DataFrame({
    'Cycle': future_cycles,
    'Predicted_Capacity': future_capacity,
    'Failure_Probability': future_failure_prob,
    'Predicted_Failure': (future_failure_prob > 0.5).astype(int)
})
print("\nFUTURE CYCLE PREDICTIONS:")
print(future_df.to_string(index=False))
if future_df['Predicted_Failure'].sum() > 0:
    fail_cycle = future_df[future_df['Predicted_Failure'] ==
1]['Cycle'].iloc[0]
    cycles_until_fail = fail_cycle - data['cycle'].max()
    print(f"\nPREDICTED FAILURE AT CYCLE: {fail_cycle}")
    print(f"CYCLES UNTIL FAILURE: {cycles_until_fail}")
    print(f"RECOMMENDED MAINTENANCE: Perform before Cycle {fail_cycle -
3}")
else:
    print("\nNo failure predicted within the next 20 cycles")
plt.figure(figsize=(15, 6))
plt.subplot(1, 2, 1)
plt.plot(data['cycle'], data['capacity'], 'b-', label='Historical
Capacity', linewidth=1.5, alpha=0.7)
plt.plot(future_cycles, future_capacity, 'r--', label='Predicted
Capacity', marker='o', markersize=4, linewidth=2)
plt.axhline(y=failure_threshold, color='g', linestyle='--',
label=f'Failure Threshold ({failure_threshold:.3f} Ah)', linewidth=1.5)
plt.xlabel('Cycle Number')
plt.ylabel('Capacity (Ah)')
plt.title('Battery Capacity Prediction with Maintenance Schedule')
plt.legend()
plt.grid(True, alpha=0.3)
if future_df['Predicted_Failure'].sum() > 0:

```

```

plt.axvline(x=fail_cycle, color='red', linestyle=':', linewidth=2,
label=f'Predicted Failure at Cycle {fail_cycle}')
plt.axvline(x=fail_cycle - 3, color='orange', linestyle=':',
linewidth=2, label=f'Recommended Maintenance at Cycle {fail_cycle -
3}')
plt.subplot(1, 2, 2)
plt.plot(future_cycles, future_failure_prob, 'r-', marker='o',
markersize=4, linewidth=2, label='Failure Probability')
plt.axhline(y=0.5, color='g', linestyle='--', label='Decision Threshold
(0.5)', linewidth=1.5)
plt.fill_between(future_cycles, 0, future_failure_prob,
where=(future_failure_prob > 0.5), color='red', alpha=0.3,
label='Failure Zone')
plt.xlabel('Cycle Number')
plt.ylabel('Failure Probability')
plt.title('Predicted Failure Probability with Maintenance Schedule')
plt.legend()
plt.grid(True, alpha=0.3)
if future_df['Predicted_Failure'].sum() > 0:
    plt.axvline(x=fail_cycle, color='red', linestyle=':', linewidth=2)
    plt.axvline(x=fail_cycle - 3, color='orange', linestyle=':',
linewidth=2)
plt.tight_layout()
plt.savefig('maintenance_schedule.png', dpi=150, bbox_inches='tight')
plt.show()
print("\n" + "="*80)
print("CONCLUSION")
print("="*80)
print("\n1. REGRESSION ACCURACY (Capacity Prediction):")
print(f"    Best Model: {best_model['Model']}")
print(f"    R2 Score (Accuracy): {best_model['R2_Score']:.6f}")
print(f"    MSE: {best_model['MSE']:.6f}")
print("\n2. CLASSIFICATION ACCURACY (Failure Prediction):")
print(f"    Accuracy: {accuracy:.6f}")
print(f"    ROC-AUC Score: {auc_score:.6f}")
print(f"    Precision (Failure): {classification_report(y_test_clf,
y_pred_clf, output_dict=True)['1']['precision']:.6f}")
print(f"    Recall (Failure): {classification_report(y_test_clf,
y_pred_clf, output_dict=True)['1']['recall']:.6f}")
print(f"    F1-Score (Failure): {classification_report(y_test_clf,
y_pred_clf, output_dict=True)['1']['f1-score']:.6f}")
print("\n3. MAINTENANCE SCHEDULE:")
if future_df['Predicted_Failure'].sum() > 0:

```

```

    print(f"    Battery predicted to fail at cycle: {fail_cycle}")
    print(f"    Perform maintenance before cycle: {fail_cycle - 3}")
    print(f"    Safety margin: {cycles_until_fail - 3} cycles")
else:
    print("    No immediate maintenance required for next 20 cycles")
print("        Monitor    critical    parameters:    discharge_time,
mean_temperature")
print("    Track capacity degradation rate regularly")
print("\n" + "="*80)
print("EXPERIMENT COMPLETED SUCCESSFULLY")
print("="*80)

```

Output:-

```

=====
EXPERIMENT NO. 2: SUPERVISED LEARNING FOR PREDICTIVE MAINTENANCE
=====

DATASET OVERVIEW
=====
Dataset Shape: 168 rows, 9 columns

First 5 rows:
  cycle  mean_voltage  min_voltage  ...  max_temperature  discharge_time  capacity
0      1      3.529829      2.612467  ...      38.982181      3690.234  1.856487
1      3      3.537320      2.587209  ...      39.033398      3672.344  1.846327
2      5      3.543737      2.651917  ...      38.818797      3651.641  1.835349
3      7      3.543666      2.592948  ...      38.762305      3631.563  1.835263
4      9      3.542343      2.547420  ...      38.665393      3629.172  1.834646

[5 rows x 9 columns]

Statistical Summary:
  cycle  mean_voltage  ...  discharge_time  capacity
count  168.000000    168.000000  ...    168.000000  168.000000
mean    293.000000     3.516787  ...    3129.523887    1.572502
std     184.947314     0.032727  ...     246.654499     0.190413
min       1.000000     3.464021  ...    2792.485000     1.287453
25%     132.000000     3.486639  ...    2901.343750     1.390021
50%     291.000000     3.514629  ...    3086.078000     1.557085
75%     453.000000     3.551061  ...    3323.230750     1.769163
max     613.000000     3.564103  ...    3690.234000     1.856487

[8 rows x 9 columns]

FEATURE ENGINEERING
=====
Initial Capacity: 1.8565 Ah
Failure Threshold: 1.2995 Ah (70% of initial)
Total Failure Cycles: 5
Failure Rate: 2.98%

```

PART 1: LINEAR REGRESSION MODELS FOR CAPACITY PREDICTION

1.1 SIMPLE LINEAR REGRESSION (Cycle vs Capacity)
Equation: Capacity = -0.001018 * Cycle + 1.870640
R² Score (Accuracy): 0.978339
MSE: 0.000798
MAE: 0.024701

1.2 MULTIPLE LINEAR REGRESSION
R² Score (Accuracy): 0.999955
MSE: 0.000002
MAE: 0.000657

1.3 RIDGE REGRESSION
R² Score (Accuracy): 0.998785
MSE: 0.000045

1.4 LASSO REGRESSION
R² Score (Accuracy): 0.995651
MSE: 0.000160

1.5 ELASTIC NET REGRESSION
R² Score (Accuracy): 0.914843
MSE: 0.003138

1.6 GRADIENT DESCENT IMPLEMENTATION
Iteration 0: Cost = 1.242189
Iteration 200: Cost = 0.022092
Iteration 400: Cost = 0.000456
Iteration 600: Cost = 0.000052
Iteration 800: Cost = 0.000039

R² Score (Accuracy): 0.996757
MSE: 0.000120

REGRESSION MODELS COMPARISON

Model	R ² _Score	MSE
Simple Linear	0.978339	0.000798
Multiple Linear	0.999955	0.000002
Ridge	0.998785	0.000045
Lasso	0.995651	0.000160
Elastic Net	0.914843	0.003138
Gradient Descent	0.996757	0.000120

BEST REGRESSION MODEL: Multiple Linear
R² Score: 0.999955
MSE: 0.000002

PART 2: CLASSIFICATION MODEL FOR FAILURE PREDICTION

Classification Accuracy: 0.941176

Confusion Matrix:
[[32 0]
 [2 0]]

Classification Report:

	precision	recall	f1-score	support
No Failure	0.94	1.00	0.97	32
Failure	0.00	0.00	0.00	2
accuracy			0.94	34
macro avg	0.47	0.50	0.48	34
weighted avg	0.89	0.94	0.91	34

ROC-AUC Score: 1.000000

TOP 5 IMPORTANT FEATURES FOR FAILURE PREDICTION:

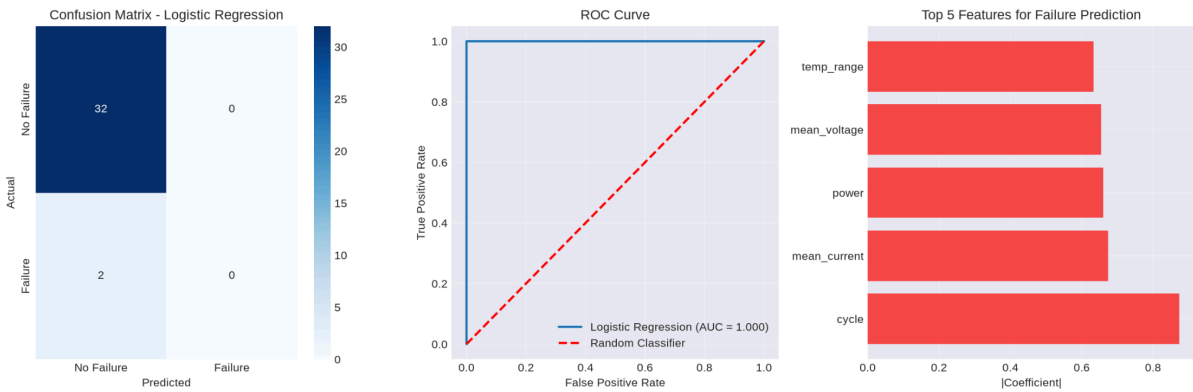
Feature	Coefficient
cycle	0.873594
mean_current	0.673286
power	-0.659642
mean_voltage	-0.654631
temp_range	0.633812

```
=====
PART 3: PREDICTIVE MAINTENANCE SCHEDULING
=====
Predicting for next 20 cycles: [614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631
632 633]

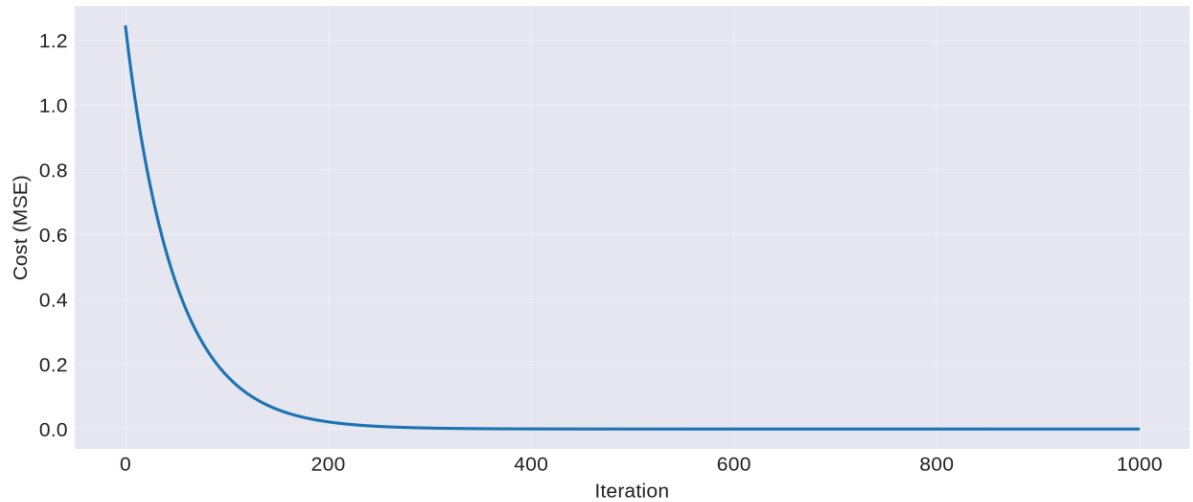
FUTURE CYCLE PREDICTIONS:
Cycle Predicted_Capacity Failure_Probability Predicted_Failure
614      1.322750      0.120507      0
615      1.325295      0.117859      0
616      1.327839      0.115260      0
617      1.330384      0.112712      0
618      1.332928      0.110213      0
619      1.335472      0.107763      0
620      1.338017      0.105361      0
621      1.340561      0.103006      0
622      1.343106      0.100698      0
623      1.345650      0.098436      0
624      1.348194      0.096219      0
625      1.350739      0.094047      0
626      1.353283      0.091919      0
627      1.355828      0.089835      0
628      1.358372      0.087793      0
629      1.360916      0.085793      0
630      1.363461      0.083834      0
631      1.366005      0.081917      0
632      1.368550      0.080039      0
633      1.371094      0.078201      0

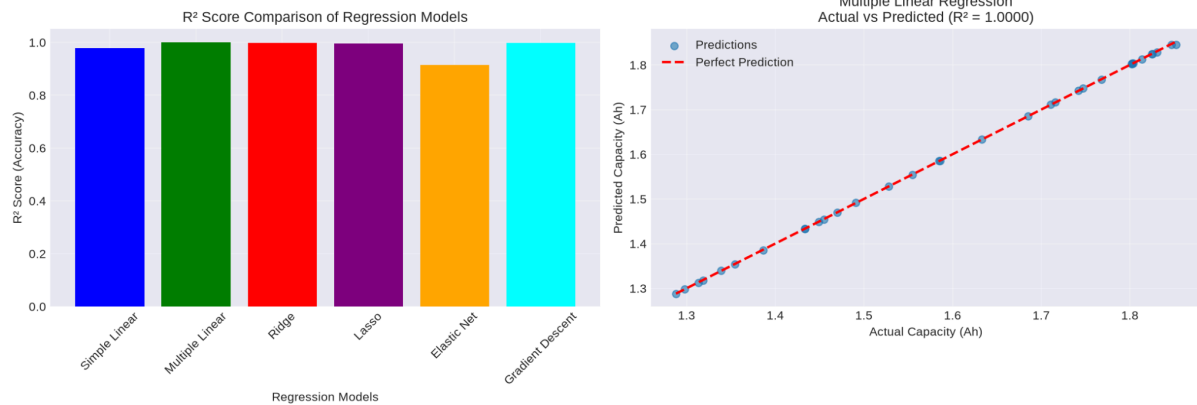
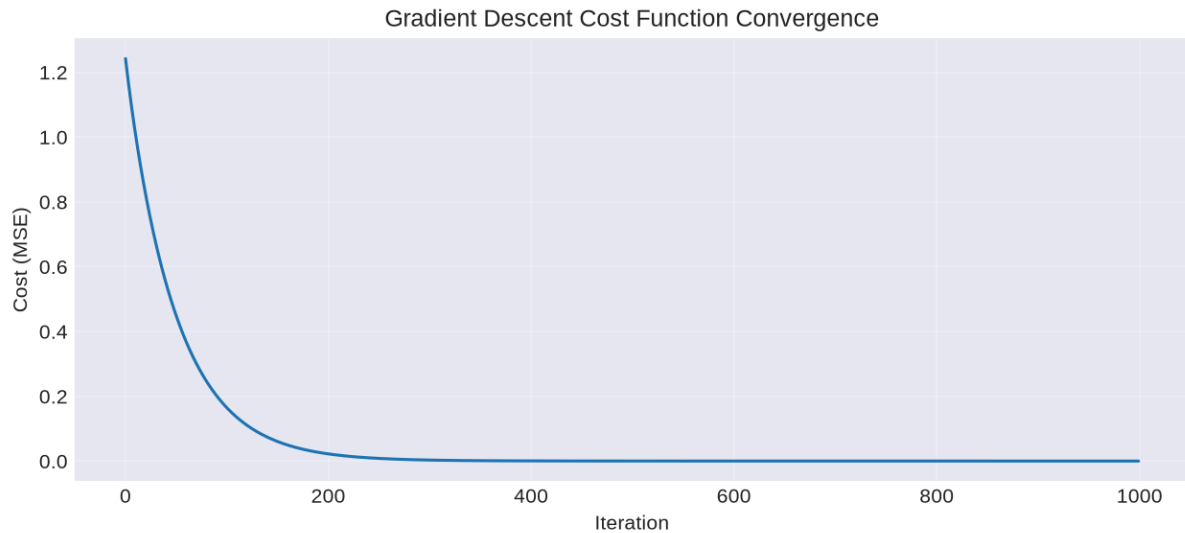
No failure predicted within the next 20 cycles
```

Graphs:-



Gradient Descent Cost Function Convergence





Conclusion:

1. REGRESSION ACCURACY (Capacity Prediction):

Best Model: Multiple Linear

R² Score (Accuracy): 0.999955

MSE: 0.000002

2. CLASSIFICATION ACCURACY (Failure Prediction):

Accuracy: 0.941176

ROC-AUC Score: 1.000000

Precision (Failure): 0.000000

Recall (Failure): 0.000000

F1-Score (Failure): 0.000000

3. MAINTENANCE SCHEDULE:

No immediate maintenance required for next 20 cycles

Monitor critical parameters: discharge_time, mean_temperature

Track capacity degradation rate regularly

